

Functional Safety Architectural Patterns for AI-Based Critical Systems

IRUNE AGIRRE and IRUNE YARZA, Ikerlan Technological Research Center, Basque Research and Technology Alliance, Spain

JAUME ABELLA and MARTÍ CARO, Barcelona Supercomputing Center (BSC), Spain

Embodied Artificial Intelligence (EAI) systems are transforming industries by enabling complex interactions with dynamic environments. However, their integration into safety-critical applications presents challenges, particularly in meeting functional safety requirements. Existing standards such as IEC 61508 and ISO 26262 provide guidance for risk mitigation but lack explicit solutions for AI-driven systems. The stochastic nature of Artificial Intelligence (AI) models, combined with increasing hardware and software complexity, introduces risks such as model insufficiencies and failures in heterogeneous computing architectures. This paper proposes a modular reference architecture for safety-critical EAI systems that aligns with functional safety and AI standards, including ISO/IEC TR 5469 and ISO/PAS 8800. Inspired by established safety patterns like the E-gas Concept in the automotive domain, the proposed reference architecture is applied to three incremental safety patterns on an NVIDIA JETSON ORIN platform, progressively addressing safety requirements while promoting reusability and compliance. This work provides a structured pathway for integrating AI into critical systems by defining a set of architectures, safety techniques, and measures that form the foundation of safety-critical systems, ensuring reliability, predictability, and compliance with functional safety frameworks.

CCS Concepts: • **Computer systems organization** → **Redundancy**; *Reliability*; **Processors and memory architectures**; Embedded software; Real-time system architecture.

Additional Key Words and Phrases: AI safety, architectural patterns, functional safety, diagnostics, monitoring

ACM Reference Format:

Irun Agirre, Irune Yarza, Jaume Abella, and Martí Caro. 2025. Functional Safety Architectural Patterns for AI-Based Critical Systems. 1, 1 (October 2025), 25 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Embodied Artificial Intelligence (EAI) refers to AI systems that are physically embedded in the real world and capable of perceiving, reasoning, and acting through continuous interaction with their environments [16]. These systems are transforming various industries by enabling advanced, context-aware interactions with increasingly complex and dynamic surroundings. Typical applications include autonomous vehicles navigating unpredictable traffic conditions, robotic surgical systems performing highly precise medical procedures, or advanced railway systems where trains equipped with AI-based perception and control technologies autonomously manage operations under variable environmental and operational conditions. In all these domains, failures can lead to severe consequences, highlighting the criticality of robust design. The development of dependable computing architectures for EAI remains a significant challenge, particularly when compliance with functional safety standards is required. Functional safety mandates

Authors' Contact Information: Irune Agirre, iagirre@ikerlan.es; Irune Yarza, iyarza@ikerlan.es, Ikerlan Technological Research Center, Basque Research and Technology Alliance, Spain; Jaume Abella, jaume.abella@bsc.es; Martí Caro, marti.caroroca@bsc.es, Barcelona Supercomputing Center (BSC), Spain.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

1

rigorous development processes, including hazard analysis, fail-safe mechanisms, and real-time monitoring to prevent accidents. Established functional safety standards, such as IEC 61508 [1] and ISO 26262 [2], provide guidance on the techniques and measures required to reduce the risk down to acceptable levels. However, in their current versions, they still discourage the use of AI in safety-critical applications and lack explicit instructions on how to ensure AI-driven systems meet safety requirements.

The integration of EAI systems into safety-critical applications introduces unique challenges due to their stochastic behavior and data-driven designs. Functional safety requires addressing both systematic and random faults, but the increased complexity of hardware and software in EAI systems complicates safety assurance. For example, the use of heterogeneous computing architectures, such as those integrating Central Processing Units (CPUs), Graphics processing units (GPUs), and specialized AI accelerators, can introduce synchronization and communication issues that may not be detected during development. Additionally, the growing size and complexity of software stacks can lead to difficulties in validating every component and interaction, increasing the risk of unanticipated failures. Moreover, EAI systems introduce additional risks, such as model insufficiencies that could lead to catastrophic consequences. For instance, an autonomous vehicle may rely on a machine learning model for pedestrian detection, but if the model has been inadequately trained on certain weather conditions (e.g., snow or fog), it might fail to recognize pedestrians in these situations, potentially leading to an accident. The techniques and measures proposed in existing functional safety standards may be insufficient to fully address these challenges. Therefore, novel architectures and mechanisms are necessary to mitigate AI-specific risks and ensure the safe deployment of AI-driven systems.

This paper addresses these gaps by proposing new architectural safety patterns that adapt to the nature of AI systems. EAI systems have applications across multiple domains, each with different levels of autonomy and, consequently, varying safety implications. The architectures that satisfy safety requirements share a significant number of commonalities across different systems with comparable safety needs. As a result, traditional safety systems rely on a range of safety patterns, which allow for the generation of specific architectures by reusing previous efforts, engineering costs and risks. One such example of a safety pattern is the E-gas Concept [8], which is used in the automotive domain and forms the basis of the architecture for many automotive systems with Automotive Safety Integrity Level (ASIL)-A or ASIL-B integrity levels as defined in ISO 26262 [2]. This paper aims to extend this concept to EAI, offering a modular and flexible reference architecture design for safety-critical EAI. This design can be tailored to specific solutions with increasing levels of autonomy and safety integrity, following an incremental strategy that bridges the current gap between safety and AI, gaining experience from less critical applications and gradually progressing towards solutions with higher criticality and autonomy.

This paper makes the following contributions. It begins by identifying the main risk factors of EAI from various sources, including functional safety and AI standards, as well as technical reports on the implications of High-Performance Embedded Computing (HPEC) platforms required for EAI inference. Based on the identified risk factors, we propose an incremental strategy to address them progressively, starting with lower-criticality applications and gradually advancing to higher-criticality ones. The core of this strategy is the proposed reference architecture, which is grounded in functional safety and AI standards such as ISO/IEC TR 5469 [14] and ISO/PAS 8800 [6]. Our architecture adopts a layered approach to modularity, promoting reusability and interoperability while ensuring compliance with functional safety requirements. Finally, we map this reference safety architecture to three different safety patterns on an NVIDIA JETSON AGX ORIN [18] platform, each with increasing safety implications for the AI subsystem.

The rest of this paper is structured as follows. Section 2 provides background on functional safety and AI. Section 3 defines the incremental strategy, including the identification of risk factors. Section 4 presents the reference safety

architecture, while Section 5 describes its application to the three safety patterns. Section 7 describes the related work. Finally, Section 8 outlines the conclusions and future work.

2 Background

This section presents background on the main concepts used in this paper: a recall of the safety standards and technical reports used as reference and nomenclature on AI-based systems.

2.1 Safety standards and technical reports

The deployment of safety-critical systems requires to comply with legal regulations in place on each country or state. This often involves a certification process where an independent certification institution must approve, through a conformity assessment, that the system is suitable and safe enough for its intended use (i.e., that it is compliant with the active legal directives). Currently, the most common certification approach is standard based, i.e., a conformity assessment is achieved by proving adherence to the applicable safety standards.

In recent years, as a result of the increasing popularity of autonomous systems, new standards are emerging to cover the safety challenges involved by the increasing level of autonomy and technologies on which they rely, such as AI.

Given that the reference safety architectural pattern is meant to be application independent, the following domain independent standards are taken as basis:

- IEC61508:2010 - Functional safety of electrical/electronic/programmable electronic safety-related systems [1]: this standard defines the requirements for the development of safety related electrical and/or electronic systems with the purpose of avoiding unacceptable risks in the system. This standard is based on the prevention of systematic errors (through rigorous process measures throughout the system lifecycle) and control and protection against random errors (e.g., by diagnostic mechanisms that move the system to a safe state). Safety functions are classified by a criticality level (i.e., Safety Integrity Level) based on their severity, frequency of exposure and controllability of the hazardous events. The higher the criticality level, more stringent are the procedures, measures, and requirements of functional safety standards. This standard proposes architectural safety patterns for traditional functional safety, which are used as the starting point in this report.

When it comes to the use of AI, IEC 61508 edition 2 only mentions artificial intelligence as a fault correction technique in the software architecture design and development (e.g., for fault forecasting, fault correction or maintenance activities supported by artificial intelligence). Despite this limitation of considering it only for fault correction, the standard does not recommend (NR) its usage in systems with an integrity level higher than Safety Integrity Level (SIL) 1. This requirement may be adapted in future editions of IEC 61508.

- ISO / IEC TR 5469:2024 - Artificial intelligence — Functional safety and AI systems [14]: this standard aims to cover the application of AI-based solutions on safety-critical systems by identifying the properties, safety risk factors, available methods and potential constraints towards the appropriate adoption of AI approaches in safety functions. This standard is of particular interest for AI-based systems development, as it covers different aspects of AI safety functions. For instance, it defines a high-level lifecycle that combines the V-model and ML lifecycle activities, it identifies the properties to be considered and evaluates the potential compliance of AI-based solutions with existing functional safety standards.

On the platform side, the standard identifies the technology elements required for ML model creation and execution and differentiates among those which can be covered by traditional functional safety techniques from

those that require further considerations. It also mentions that GPU based systems may have special failure modes to be addressed and some architectural considerations are proposed (like the use of supervisors, redundancy and diversity and detection mechanisms).

Moreover, this standard defines different DL usage levels (see Section 2.2), which is used as the basis for the incremental strategy of Section 3 and the “Architectural safety patterns for AI” on which the reference architecture of Section 4 relies on.

In addition to these domain independent standards, technical reports from different domains are also taken as reference:

- CAST-32A and AMC-20-193 from the avionics domain for multicore processors [7, 9]: these two technical reports from the avionics domain provide guidance on the adoption of multicore processors for critical systems. They recommend the practices to be considered when working with multicore processors, including considerations for dynamic allocation and multicore interference mitigation. Many of the principles described there are also applicable to HPEC platforms that integrate multicore processors together with additional accelerators and diverse computing resources.
- E-Gas architecture concept from the automotive domain [8]: The E-Gas architecture concept, defined by the German Association of the Automotive Industry (VDA), has the aim to standardize the safety architecture for engine control systems. The defined architecture is in compliance with ISO 26262 and it can be applied as a well-trusted design principle. The E-Gas defines a 3-level monitoring concept that has been used as reference in Section 4.2.2), adapting it to AI systems particularities and to ISO/IEC TR 5469.

2.2 AI-based safety system

The ISO/IEC TR 5469 framework and the European Union Aviation Safety Agency (EASA) concept paper propose several approaches for the classification of deep learning (DL) usage in a system. ISO / IEC TR 5469 describes different DL usage levels:

- DL usage Level A1: AI technology is used and it is possible to make automated decision of the system function using AI technology.
- DL usage Level A2: AI technology is used but it is not possible to make automated decision of the system function using AI technology (e.g., AI technology is present in the system for diagnostics).
- DL usage Level C: AI technology is not part of a safety function but can have an impact on it.
- DL usage Level D: AI technology is not part of a safety function and does not have an impact on it due to sufficient segregation and behaviour control.

Similarly, the EASA concept paper [10] classifies AI systems based on the level of automation / assistance to the user:

- EASA Level 1: AI used for assistance to human (human augmentation or cognitive assistance in decision and action selection).
- EASA Level 2: Human-machine teaming (cooperation or collaboration).
- EASA Level 3: Autonomous AI-based decisions and actions.

3 Risk-based Incremental Strategy

In order to define the safety architecture patterns, first main problems arising from the use of EAI in safety critical systems and their root causes are analysed and then an incremental strategy to deal with them is defined.

3.1 AI Risk factors

As for traditional safety related systems, the main risk factors related to AI-based systems can be originated either in its development or at operation. Fig. 1 depicts this classification criteria together with information sources used in this section to identify the different risk factors. During the development, there are several stages at which systematic faults can originate, like incomplete specifications, biased training data or wrong design decisions affecting the ML model. In order to reduce the probability of systematic faults in the AI-based subsystem, safety considerations shall be taken in the AI lifecycle. Development related risks should be addressed by the proper application of Functional Safety Management (FSM) methodologies, as that defined in [11] for AI-based critical systems. However, the high complexity of AI systems makes it very difficult to mitigate all systematic faults through design and verification and validation processes, and it is assumed that residual systematic faults will manifest at operation and should be covered by the runtime mechanisms in the safety architecture too, as done for random faults.

At operation time, residual systematic faults and random faults can impact different elements of the AI-subsystem, this determines the architectural measures required for runtime error detection and monitoring. Based on ISO/IEC TR 5469 we classify these AI technology elements in the 5 groups shown in Fig. 1:

- (1) Application Independent: The lowest levels of the AI technology elements can be considered as application independent:
 - Computational Hardware: refers to the hardware platform and its processing units (e.g., CPU, GPU, Accelerators).
 - Operating System (OS) / middleware: This component is not included in ISO/IEC TR 5469, but many faults could also originate on this layer.
 - Libraries: Machine Learning (ML) components often require specific libraries for their inference (e.g., CUDA Deep Neural Network (CuDNN)). These libraries can also be implemented in different programming languages such as Compute Unified Device Architecture (CUDA), C, C++, python...
- (2) Application dependent: Higher levels of the AI technology elements are usually platform dependent:
 - ML framework: many different frameworks can be used depending on the application that is being implemented (e.g., TensorFlow, Pytorch, Keras).
 - ML model: the ML model will depend on the specific application and the design decisions taken for it (architecture, hyperparameters, number of layers, etc).

This paper has been based on the following information sources to identify the relevant risk factors (see Fig. 1):

- Functional safety standards: Functional safety standards such as IEC 61508, ISO 26262 or EN 5012x only address the application independent technology elements from Fig. 1 (computational hardware and software) as they do not address AI-specific topics. Traditional functional safety standards classify failures based on their origin as systematic (i.e., failure, related in a deterministic way to a certain cause) or random (i.e., failure, occurring at a random time)
- Technical reports on HPEC: CAST-32A / AMC-20-193 focus on multicore processors and therefore mostly on the hardware computing element and allocation of resources to deal with interferences. The following main risk factors are considered: contention for resources and interference between software applications or tasks, interference caused by the arbitration of shared resources, verification of the use of shared resources, exceedance of resource capabilities by software applications, hardware dynamic features that can alter system behaviour, and configuration settings.

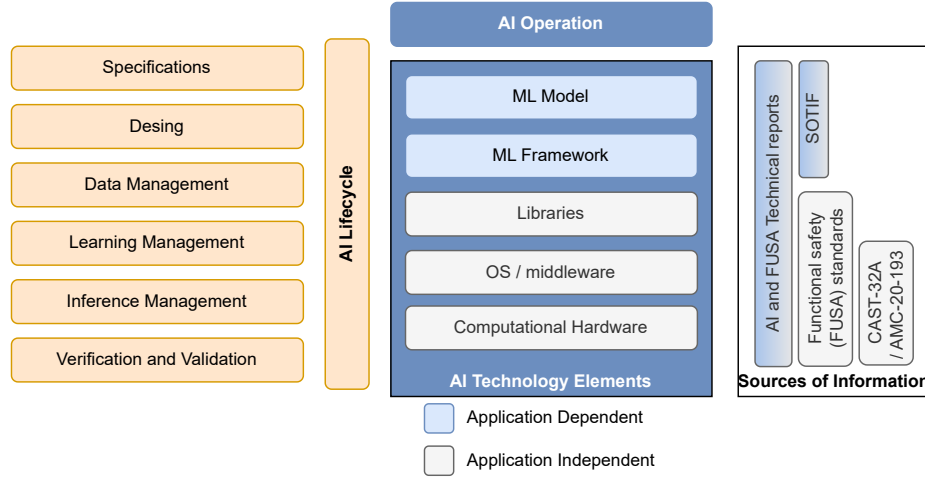


Fig. 1. EAI Risk factor classification and sources of information

- AI and Functional safety technical reports and standards: ISO/IEC TR 5469 defines the following properties for AI application dependent technology elements: degree of automation and control, degree of decision transparency and explainability, environmental complexity and vagueness of specifications, adversarial inputs, AI Hardware issues, and technological maturity.
- Safety Of The Intended Functionality (SOTIF): ISO/DIS 21448 defines the concept of functional insufficiencies: insufficiency of specification (specification possibly incomplete), and performance insufficiency (limitation of the technical capability).

Tab. 1 summarizes the risk factors, mapping them to the affected AI-technology elements the source of information.

Risk factor	AI technology element	Source
Traditional FUSA risk factors (systematic / random)	All AI technology elements (i.e., computational hardware, OS / middleware, libraries, ML framework and ML model).	IEC 61508 and derived FUSA standards (ISO 26262, EN 5012x, ...)
HPEC platform integration risk factors	Computational hardware and lower-level SW (i.e., OS / middleware).	CAST-32A / AMC-20-193 ISO 26262 part 11
AI performance insufficiency	ML model or framework.	ISO/DIS 21448 (SOTIF)
AI & FUSA risk factors (low/medium integrity level)	All AI technology elements (i.e., computational hardware, OS / middleware, libraries, ML framework and ML model).	AI and FUSA Technical Reports (ISO/IEC TR 5469)

Table 1. Summary of risk factors and mapping with technology element and standards

3.2 Incremental strategy

In light of the uncertainty and the early stage of AI adoption in critical systems, this work follows the incremental strategy outlined in Fig. 2. This approach aims to enhance the safety implications of the AI subsystem progressively as operational experience, that is, real-world field data and lessons learned from the actual use of AI in safety-critical applications, is accumulated over time. Through this strategy, the risk factors identified in Table 1 are addressed incrementally.



Fig. 2. Incremental strategy for AI adoption in safety critical systems

The proposed incremental strategy consists of three SPs and is aligned with the ISO/IEC TR 5469 framework and the classification of deep learning (DL) usage levels detailed in the EASA concept paper:

- Safety Pattern 1 (SP1) (described in Section 5.2) aims to address up to a Deep Learning (DL) usage level D or EASA Level 1. The AI/ML constituent is not part of the safety function, therefore, we set the focus on safety techniques and measures to detect and mitigate errors associated with traditional functional safety and HPEC platform integration risk factors.
- Safety Pattern 2 (SP2) (described in Section 5.3) aims to address up to a DL usage level C or EASA Level 2. The AI/ML constituent is not part of the safety function although it collaborates on the decision and can therefore have an impact on it, so we assume that the AI/ML constituent has a low/medium SIL. This Safety Pattern (SP) considers the techniques and measures of SP1 and sets the focus on solutions to detect and mitigate errors associated with AI performance insufficiency as well as AI and functional safety risk factors on the ML framework and DL model.

- Safety Pattern 3 (SP3) (described in Section 5.4) aims to address up to a DL usage level A1 or EASA Level 3, where the AI/ML constituent is part of the safety function. The AI/ML constituent provides autonomous AI-based decisions and actions, so it has a high SIL. SP3 considers the techniques and measures from previous SPs and sets the focus on achieving higher integrity level.

4 Reference Safety Architecture Pattern

This section defines a reference safety architecture pattern for AI-based critical systems with the goal of providing a baseline design that can later be tailored to different use cases, considerably reducing the design effort, costs, and risks.

4.1 Scope

The reference safety architecture pattern is designed for systems that meet the structure depicted in Fig. 3. This is based on the assumption that the AI-based critical system consists of two main subsystems:

- (1) AI-based subsystem. Comprises all the AI-related elements in the architecture. This includes the *AI/ML Constituent*, which consists of the ML model as well as the corresponding data pre-processing and post-processing components.
- (2) Traditional subsystem. Comprises the components usually present in a traditional (non-AI based) system. This traditional subsystem can be either:
 - Functions complementary to the ML constituent, required to perform the function required by it.
 - Fallback subsystem: Following known proper functional safety precautions, a safe back-up function to the ML component might be implemented within the non-AI subsystem. This back-up decision system will take over the system in the event that a functional safety problem is detected, ensuring that the AI system does not cause harm.

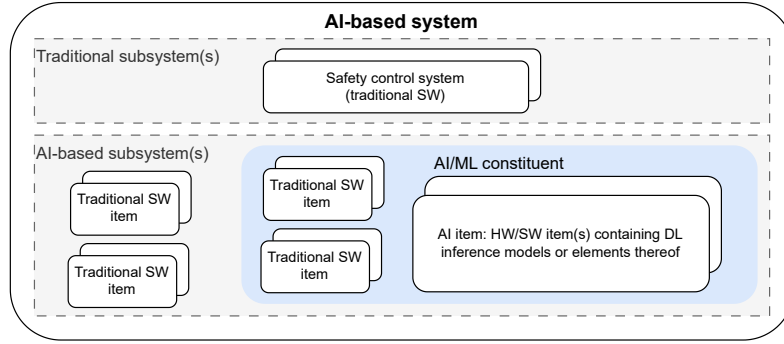


Fig. 3. Scope of the reference safety architecture patterns [10].

4.2 System architecture and techniques

Fig. 4 shows the reference safety architecture pattern where the main safety elements are illustrated by means of enumerated rhombus symbols. These elements constitute the main building blocks to define the specific safety architectural patterns for each DL usage level. In summary, the reference architecture pattern of Fig. 4 integrates the following three safety approaches on it:

- (1) Diverse redundancy within the AI/ML constituent (Subsection 4.2.1). Note that redundancy and diversity are commonly also applied to traditional systems; however, we consider such part out of the scope of this research work, and we just focus on its application to the DL component.
- (2) Diagnostic and monitoring mechanisms (Subsection 4.2.2). A hierarchical diagnostic and monitoring concept consisting of three levels (L1 to L3) is defined.
- (3) Supervision function (Subsection 4.2.3). Its main goal is to check the appropriateness of the environment and to supervise the output of the ML constituent to identify unsafe situations and establish the limits for safe operation, providing a safe envelope. In addition, this subsystem includes runtime AI explainability techniques to improve model interpretability and help understand how the models make the predictions during their actual operation.

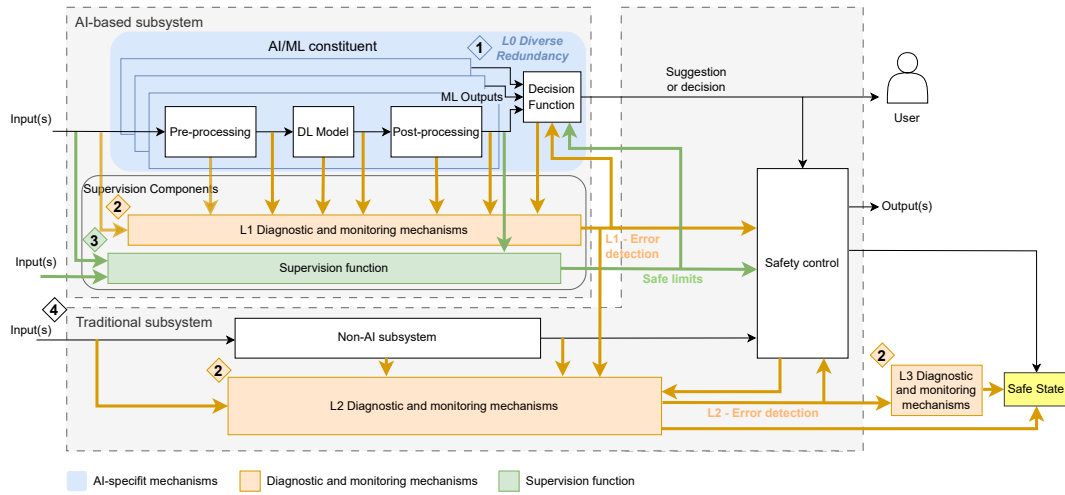


Fig. 4. Reference safety architecture pattern for safe AI-based systems.

4.2.1 Diverse Redundancy. The goal of redundancy and diversity on the DL model is to enhance the performance and to increase the portion of AI subsystem errors that can be detected and controlled. In addition, redundancy can also improve system availability, by maintaining system operation even when one instance of the redundant architecture is failing. Redundancy shall be complemented with diversity in order to deal with systematic faults and model insufficiencies. When used for improved diagnostics, diverse redundancy can be considered as a particular technique of the AI subsystem diagnostic mechanisms of Subsection 4.2.2. We base on the following three types of diverse redundancy that can be combined to build the Diverse Redundancy Schemes (DRS) proposed in Table 2 with varying degrees of diagnostic coverage:

- Inference Platform diversity (identified as “I” in Table 2) – different platform resources for running the inference. Inference Platform diversity can be applied to different platform elements. These are some example techniques:
 - Inputs (e.g., diverse cameras, sensors, input image flips...)
 - Processing resources (accelerators, CPUs...)
 - OS, execution framework (e.g., TF lite, pytorch, darknet...)

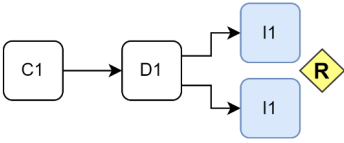
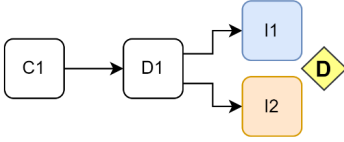
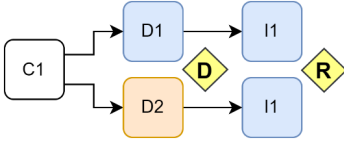
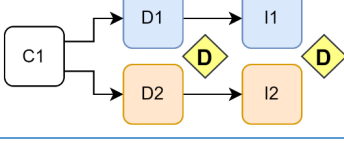
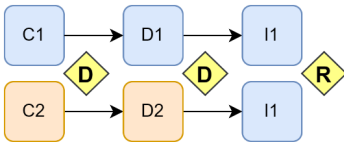
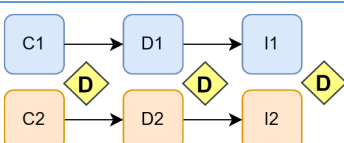
- Temporal (e.g., different time instants)
- Data precision (e.g., float vs double, short vs int vs long int)
- Data rounding (e.g., for floating point data)
- Sources for potential random parameters
- DL model Development diversity (identified as “D” in Table 2): different techniques or elements for developing the DL model. DL model development diversity can be applied to different development phases and elements. These are some example approaches:
 - Model Architecture
 - Hyperparameters
 - Prediction criteria (e.g., confidence threshold, bounding box overlapping threshold, single vs multi class prediction...)
 - Training datasets
 - Training process (including convergence thresholds, weighting predictions...)
 - Training platform
- Concept diversity (identified as “C” in Table 2): different problem formulation with same final goal. The problem formulation may vary depending on the application are. These are some example approaches:
 - Object detection vs object part detection
 - Object detection vs obstacle free path detection

Decision Function: Note that, differently to usual control algorithms where diverse redundancy is applied preserving bit-level precision (e.g., dual-core lockstep), some diversity schemes lead naturally only to semantic-level precision, meaning that we expect the same result in semantic terms (e.g., same object detection in the same location), but results are very likely to be different at bit level (e.g., different confidence values, not fully identical bounding boxes, etc.). Therefore, mechanisms to produce the final (combined) prediction or report an error are needed, such as voting, averaging, and Non Maximum Suppression (NMS).

Before introducing those decision functions, it is important to determine the criteria to use to decide what data needs to be combined to provide a single prediction. In some cases, DL models may predict a single feature, making it obvious what should be combined. However, this is not always the case. For instance, in the case of camera-based object detection, objects may come along a confidence level, a class, and a bounding box. The usual case to determine whether two objects detected by diverse DL models correspond to the same object in this case is doing so only when both of them belong to the same object class (e.g., a car), and the bounding boxes overlap enough. The most common metric used to determine whether two bounding boxes overlap enough is the Intersection Over Union (IoU) metric [19] (eq. 1):

$$\text{IoU} = \frac{\text{Area}_A \cap \text{Area}_B}{\text{Area}_A \cup \text{Area}_B} \quad (1)$$

Where AreaA and AreaB correspond to the area of the two objects to be combined. Note that, upon building on higher degrees of redundancy (e.g., three or more redundant instances), IoU can either be applied globally to the objects across the different redundant instances, or obtained using any other criteria to decide such as, for instance, using an object as reference, and computing the IoU of each other object against the reference one. For those objects regarded as being the same object, different functions can be realized to combine the output of different predictions, each one with its own pros and cons, hence, being some of them particularly suitable for specific problems. While one could devise an arbitrary number of such functions, hereafter we describe some relevant examples that allow illustrating the

ID	Type of Diverse redundancy: Concept – Development – Inference	Diagnostic Coverage	Description
DRS_1		Low	Single DL model development, replicated in the inference platform without diversity (each replica uses same SW stack and processing elements (e.g., GPU)). Allows addressing traditional FUSA risk factors.
DRS_2		Low to medium	Single DL model development, replicated in the inference platform with diversity (each replica uses different resources (different inputs, AI framework, OS, processing elements (e.g., GPU and CPU), etc.)). Allows addressing traditional FUSA risk factors.
DRS_3		Low to medium	Development of two different DL models based on same concept (e.g., object detection) with diversity in the training process, training data, model architecture etc. Inference without diversity (each replica uses same SW stack and processing elements (e.g., GPU)). Allows addressing AI & traditional FUSA risk factors as well as AI performance insufficiencies.
DRS_4		Medium to high	Combination of DRS_2 and DRS_3 with DL model Development diversity and Inference Platform diversity. Allows addressing AI & traditional FUSA risk factors as well as AI performance insufficiencies.
DRS_5		Medium to high	Development of two different DL models based on different concepts (e.g., one for object detection and the other for obstacle free path detection). This requires DL model Development diversity. Inference without diversity (each replica uses same SW stack and processing elements (e.g., GPU)). Allows addressing AI & traditional FUSA risk factors as well as AI performance insufficiencies.
DRS_6		High	Combination of DRS_4 and DRS_5 with concept diversity, DL model Development diversity and Inference Platform diversity. Allows addressing AI & traditional FUSA risk factors as well as AI performance insufficiencies.



Redundancy



Diverse Redundancy

Table 2. Diverse Redundancy Schemes

different tradeoffs to take into account. Moreover, since predictions may include multiple data, those decision functions may come along with specific characteristics when applied to different data (e.g., when applied to confidence values or bounding boxes). Some key functions to consider are as follows:

- **Averaging:** this function can be applied to any number of outputs and different types of data. For instance, it can be used for dual (DMR) and triple modular redundancy (TMR), and can be applied to the confidence values and bounding boxes in the case of camera-based object detection. Such a function avoids the risk of sticking to the worst prediction, but on the other hand guarantees that the worst prediction will influence the final prediction, since it will have a fair share of impact on the outcome.
- **Voting:** This function is typically applied when there are at least three redundant instances and, preferably, an odd number of them. In such cases, the majority decides (e.g., whether an object exists). Once a decision is made, the parameters are usually taken from the model most aligned with that outcome. For example, in TMR, if two models detect an object and one does not, the system concludes that the object is present and selects the prediction from the deep learning model with the highest confidence. While not suitable for DMR, this strategy allows leveraging the most confident model when there is majority agreement. However, it also carries the risk of choosing a poor-quality prediction if confidence does not reflect accuracy. When using voting with an even number of instances (e.g., DMR), a tie-breaking rule is required. One option is to accept a prediction if at least one model supports it—similar to the NMS approach discussed next. Alternatively, ties can be resolved by discarding the prediction, which reduces false positives but increases the chance of missing correct detections.
- **NMS:** this function performs conceptually a union of the predictions just merging those predictions that correspond to the same object. Hence, it is a suitable scheme when the redundant instances to be combined tend to miss objects in the case of camera-based object detection, or when it is more important finding true objects than filtering out false detections since as long as one instance detects an object, NMS will preserve it. For obvious reasons, this trend towards over detection of objects exacerbates as we increase the number of redundant instances to be combined.

4.2.2 Diagnostic and Monitoring mechanisms. The reference safety architecture pattern is based on the hierarchical diagnostics and monitoring approach of Fig. 5. This strategy seeks to decouple AI specific techniques from traditional functional safety approaches and to ease, in this way, the later tailoring to incremental safety patterns.

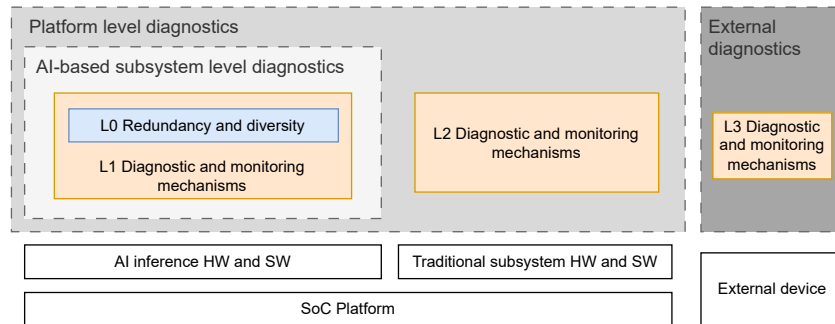


Fig. 5. Hierarchical diagnostics and monitoring approach

The proposed approach is based on four diagnostic levels ($L0$ to $L3$) to incrementally cover the errors on the following subsystems:

- (1) AI-based subsystem level diagnostics ($L0 - L1$): The redundancy and diversity ($L0$) described in previous subsection is complemented by additional diagnostic and monitoring mechanisms ($L1$) with the purpose of detecting runtime errors or model insufficiency and anomalies on the AI subsystem and the elements required for its execution (e.g., accelerators, AI frameworks, etc.). Note that $L1$ diagnostics and monitoring mechanisms may, in turn, include AI components (e.g., to assess suitability of the data used for inference, or monitoring abnormal results from specific Deep Neural Network (DNN) layers). All in all, the $L1$ mechanisms have the following purpose:

- Diagnose the AI model to detect runtime errors or model insufficiencies and anomalies.
- Diagnose the elements that participate in AI inference: complex HW and SW stack, black / grey box diagnostics.
- Monitor the use of resources at the AI subsystem level to guarantee freedom from interference at platform level ($L2$).

These mechanisms can be applied on different elements of the AI-subsystem. Bellow we present different $L1$ diagnostic mechanisms that may be applied to the different AI-subsystem elements.

- Inputs: diagnostic mechanisms for input correctness, data quality, data redundancy, temporal consistency... Examples include Anomaly detectors or Input image comparison (e.g., consecutive input frames, input from redundant cameras).
 - Model: diagnostic and monitoring mechanisms for execution errors, timing, program sequence, neuron activation patterns, execution signatures...
 - Outputs: diagnostic mechanisms for outputs, plausibility checks, input-output correlation, temporal consistency, trajectory prediction...
 - Resource usage: monitoring mechanisms for resource usage (e.g., CPU/GPU usage, memory usage)
- (2) Platform level diagnostics ($L2$): This includes $L0 - L1$ diagnostics and additional $L2$ diagnostics to detect runtime errors on additional platform HW and SW components following traditional functional safety practices and diagnostics techniques (e.g., memory self-tests, freedom from interference at platform level...). However, traditional functional safety standards do not reflect the increasing complexity of emerging systems, therefore, L2DM mechanisms shall integrate advanced diagnostic approaches for high-performance platforms as well as the for the software applications that run on them.
 - Traditional functional safety diagnostics. Traditional functional safety standards such as IEC 61508, ISO 26262 and EN 5012x define the faults or failures that shall be considered to reach different degrees of Diagnostic Coverage (i.e., Low (60%), Medium (90%) or High (99%)) for the different system components and proposes the applicable diagnostic techniques for each: electromechanical devices, discrete hardware, bus, CPU, memories, clock, communications, sensors, final elements, etc.. The defined diagnostic strategy will comprise among others, startup diagnostics, periodic diagnostics, as well as forced checks of the safety function. On the software side, functional safety standards propose techniques and measures to prevent systematic failures on the different stages of the development life-cycle (e.g., requirements specification, design and development, testing and integration...). For each technique or measure, the standard gives a recommendation (highly recommended (HR), recommended (R), not recommended (NR)) according to the target safety integrity level. Additionally, when the software implements safety functions of different safety integrity levels, it shall be

demonstrated either that independence of execution is achieved both in the spatial and temporal domains, or that any violation of independence is controlled.

- Techniques for multicore / high-performance platforms. Based on CAST-32A [7] and AMC 20-193 [9], the following four principles should be met:
 - (a) Determining the final configuration. The designer shall determine which is the intended final configuration that will enable to satisfy system requirements and this configuration shall be protected against modification at runtime.
 - (b) Managing interference channels. It is required to identify interference channels in the intended final configuration and to define the means to either avoid interference by design or upper-bound it so that timing deadlines are not exceeded. Upper-bounding interference involves analysing the use of shared resources and designing the means to control contention.
 - (c) Verifying the use of shared resources. Resource usage and data and control flows among cores shall be verified by guaranteeing that the software does not exceed the use of available resources even in worst-case scenarios.
 - (d) Error Management. The system shall include features for multicore specific error detection and handling.

In order to achieve these four principles, we define techniques and measures to ensure freedom from interference at platform level, with the support of the following L2 diagnostic and monitoring (L2DM) mechanisms:

- L2DM Configuration check: The configuration shall be defined and verified during system development process. Then, at runtime, L2DM checks that this configuration is kept (e.g., by a Cyclic Redundancy Check (CRC) check).
- L2DM Interference mitigation and control: This L2DM mechanism addresses both “managing interference channels” and “verifying use of shared resources” principles from CAST-32A / AMC 20-193. L2DM is the responsible to check that all critical platform SW components meet with their deadline and if this is the case, it refreshes an external watchdog (*L3*). In this way, if any critical software component violates a timing deadline or if the platform is not able to run *L2* diagnostics, the *L3* external device will be able to detect it. To this end, it is required that *L2* knows the deadline of each critical software component, which shall be determined at design time by Worst Case Execution Time (WCET) analysis.
- L2DM Health Management: L2DM includes an error management module to react to different platform level errors. To this end, L2DM collects the results from *L0* and *L1* diagnostic and monitoring mechanisms. Whenever *L0*, *L1* or *L2* diagnostics and monitoring detects an error in the platform, either by startup diagnostic tests or periodic checks, L2DM is the responsible of triggering the required reaction. This will depend on the specific error and the application itself. For instance, in a Triple Modular Redundancy (TMR) architecture if *L0* determines that one of the redundant instances is providing incorrect inputs, the system could keep working for a limited time in a degraded mode with the other 2 active redundant instances. Similarly, if the interference mitigation and control of L2DM determines that a contender is using a shared resource more than planned in the design, the action could be to force it to stop so that it does not cause uncontrolled interferences to the critical component. In many other cases, the reaction could be to move the system to the safe state. In addition, the health monitoring in L2DM shall provide an interface with *L3* diagnostic and monitoring (L3DM) (e.g., refreshing a watchdog), in such a way that if there is a problem in the platform that affects the correct execution of L2DM, the external L3DM will be able to detect it.

- (3) External diagnostics (*L3*): Following traditional functional safety practices some diagnostic may be implemented on an external device such as an external watchdog or microcontroller. Moreover, the *L3* diagnostics and monitoring implements error management for the lower-level diagnostic and monitoring mechanisms (i.e., *L0*, *L1* and *L2* mechanisms).

4.2.3 Supervision function. The supervision function applies elements of control theory to minimally bound AI operations. All in all, it shall control that the AI system works within a predefined safety envelope. To this end, the supervisor function shall determine a safe subset of the action space or safe envelope (prior to the execution or at runtime) and compute a set of constraints or limits. The supervision function comprises techniques such as:

- A non-AI safety function that outputs an acceptable range of outputs for a given input and limits the intelligent control output (this approach might not be appropriate for systems with dynamics).
- Defining minimal bounds though control theory methods (e.g., barrier functions approaches).
- Checking functions such as metrological self-check or self-validation.

In addition, the supervision function may implement explainability techniques to enhance understanding of the DL model's behavior, as well as its decisions, recommendations, or predictions. The purpose and level of rigor of these explanations vary depending on both the DL usage level and the intended stakeholder, as further elaborated in the next section for each SP. A more comprehensive study of explainability techniques suitable for integration at runtime as part of the supervisory components is provided in [17]. Examples of such techniques include:

- Uncertainty management to detect known uncertainties and unknown situations
- DL model anomaly checks
- Intrinsic explainable surrogate models designed to approximate and interpret the behavior of the original DL models
- Cross-model interpretability checks, ensuring consistency of explanations across different DL models (e.g., in a TMR setup) and across different explanation techniques
- Trustworthiness scoring on the decision function, which can support selection of the most reliable output by combining insights from model ensembles and explanation-derived metrics (e.g., trustworthiness score)

5 Safety Patterns

Following the incremental strategy described in Section 3, this section presents the mapping of the reference safety architecture pattern (see Fig. 4) to the different DL usages levels. In particular, three Safety Patterns are defined:

- (1) Safety Pattern 1 (*SP1*) (for DL usage level D / EASA Level 1).
- (2) Safety Pattern 2 (*SP2*) (for DL usage level C / EASA Level 2).
- (3) Safety Pattern 3 (*SP3*) (for DL usage level A1 / EASA Level 3).

The following subsections present, on the one hand, the NVIDIA Jetson AGX Orin platform, which has been selected as the target platform for the adoption of the SPs. Moreover, for each Safety Pattern, the proposed system architecture and safety techniques, as well as its application on the target platform (i.e., NVIDIA Orin) are presented. The description of the Safety Patterns follows an incremental approach; therefore, each Safety Pattern comprises also the safety techniques of the previous one.

5.1 NVIDIA Jetson AGX Orin

The NVIDIA Orin comprises 3 clusters of 4 Arm Cortex-A78AE CPUs, a dual core Sensor Processing Engine (SPE) which can be configured in lockstep, a NVIDIA Ampere GPU, ad-hoc AI-oriented accelerators, and video encoder/decoder.

Clusters based on Arm cores provide general purpose computing capabilities suitable for control software and software components around AI-based software, but without needing AI-specific support or massively parallel hardware components. Each of the three clusters includes four cores that can be used as either 4 independent cores, or as 2 pairs of lockstep cores. Cores include local (L1 and L2) caches. There is a L3 cache shared across all cores in the cluster, but it includes partitioning support. We refer to the set of 3 core clusters as CCPLX for short.

Computing devices like the GPU and the AI-oriented accelerators have a complementary nature to the core clusters and are suitable for AI software and other embarrassingly parallel software. In particular, the GPU has a number of Stream Multiprocessors (SMs) sharing a L2 cache, which, apparently, cannot be partitioned.

Core clusters and the GPU share an L4 cache that, so far, cannot be partitioned, and hence, both cores and the GPU share it. However, we note that, due to its limited size, does not seem to be intended for providing increased cache space for the L3 caches of the core clusters or the L2 cache of the GPU.

5.2 Safety Pattern 1

On this safety pattern, the AI subsystem aids the driver or person responsible of the safety of the system. However, the AI subsystem is not part of the safety function. On the following subsections we present the system architecture for *SP1* as well as the corresponding diagnostic and monitoring mechanisms.

5.2.1 System Architecture. *SP1* assumes the integration of safety related software (i.e., non-AI subsystem, safety control, safety diagnostics) and non-safety related software (i.e., AI/ML constituent, supervision function) within the same System on Chip (SoC) on a HPEC platform. The main components of the reference safety architecture are applied as follows in *SP1*:

- **Diverse Redundancy:** In this safety pattern, the safety function is implemented by the non-AI subsystem and the safety control, therefore, the AI/ML constituent has no safety implications. Given that, diverse redundancy is not required in the AI/ML constituent.
- **Diagnostic and monitoring mechanisms:** Even if the AI/ML constituent is not safety related, in *SP1* the main challenge rests on the integration of both the safety software together with the software involved in the inference of the DL model and to guarantee freedom from interference among them. Therefore, L1DM is the responsible for monitoring resource usage and timing of the AI/ML constituent, and L2DM / L3DM to apply the required diagnostic mechanisms on the safety subsystem and guarantee freedom from interference. The application of the diagnostic and monitoring strategy in *SP1* is further elaborated in Subsection 5.2.2.
- **Supervision function:** In *SP1*, the supervisor has no safety implications. It can provide explanations to the final user (e.g., a driver) by providing runtime information such as heatmaps.
- **Traditional subsystem:** in *SP1* the safety function is performed by traditional SW components. No fallback subsystem is considered.

5.2.2 Diagnostic and monitoring mechanisms. *SP1* applies the hierarchical diagnostics and monitoring approach as follows:

- *SP1-L1DM*: As the AI subsystem is not safety related, there are no specific diagnostics for it. However, the *L1* monitoring mechanisms are necessary to guarantee the required freedom from interference at platform level. In this particular case, the monitoring of the use of shared resources among the safety and non-safety software components (i.e., memory, caches, interconnect) is of special interest. In *SP1*, *L1DM* will collect such events for the non-safety AI subsystem and provide the information to the *L2DM*.
- *SP1-L2DM*: The *L2DM* mechanism implements traditional functional safety techniques and measures according to the recommendations on functional safety standards. These diagnostics comprise start-up as well as periodic checks that can be applied to the different subsystems that participate in the execution of the safety function (e.g., sensors, CPU, memories, clock, electromechanical devices, etc.).

In addition, *L2DM* is responsible for checking system configuration and providing the mechanisms for interference mitigation and control. As introduced in Section 1.1.2, the NVIDIA Orin platform provides a number of diverse processing resources (CPU Complex (CCPLEX) core clusters, SPE, GPUs...) with multiple configuration options. The allocation of such resources to software components and their configuration play a crucial role in the mitigation of interference required by *SP1*. Fig. 6 and Table 3 present different resources and configuration options that could be adopted for *SP1*.

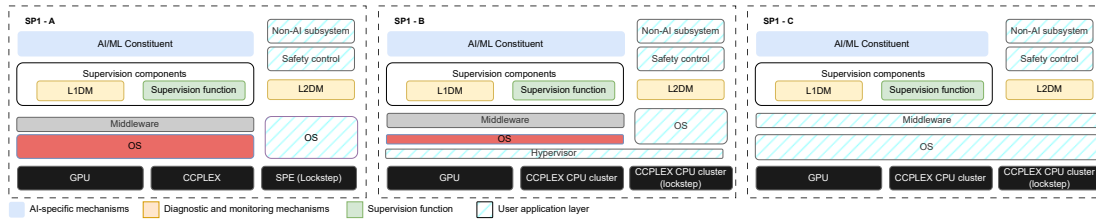


Fig. 6. *SP1* definition and allocation to NVIDIA Orin resources and configuration options

The three *SP1* configuration options rely on mechanisms provided by the platform, which include cache partitioning features in the CCPLX, and performance monitoring counters in the CCPLX allowing to monitor resource usage in the CCPLX.

- Independent computing components: The NVIDIA Orin provides abundant computing resources, the safety critical tasks can be allocated either to the SPE or to a specific CCPLX cluster, increasing independence and diversity among safety and non-safety tasks.
- Spatial independence. As explained before, the NVIDIA Orin platform provides explicit support to enable spatial independence in the form of Memory Management Unit (MMU)s. Therefore, spatial independence can be achieved by design with usual operating system support.
- Temporal independence. Safety related tasks needing temporal independence can be executed in the SPE or CCPLX. The degree of independence achieved across tasks running in the same CCPLX core cluster is relatively high if L3 cache partitioning is used, and even higher across tasks running in different core clusters as proposed in *SP1-B* and *SP1-C*. The remaining interference can be measured with existing event monitors so that safety measures can be built at software level if measured interference reaches system or application-specific thresholds.

SP1 Element	Safety / non-safety	SP1 - A NVIDIA Orin resources and configuration	SP1 - B NVIDIA Orin resources and configuration	SP1 - C NVIDIA Orin resources and configuration
AI/ML constituent	Non-safety AI based SW	CCPLEX CPU Cluster (Cortex A78)	CCPLEX CPU Cluster (Cortex A78) – different CPU cluster for safety SW	CCPLEX CPU Cluster (Cortex A78) – different CPU cluster for safety SW
Supervision components	Non-safety traditional or AI based SW	GPU for AI inference Memory controller fabric and traffic from CPU cluster to GPU	GPU for AI inference Memory controller fabric and traffic from CPU cluster to GPU	GPU for AI inference Memory controller fabric and traffic from CPU cluster to GPU
Decision function	Non-safety traditional SW	MMUs for spatial independence SAFEXPLAIN SW Stack	MMUs for spatial independence L4 cache partitioning or disabled SAFEXPLAIN SW Stack separated from safety SW by hypervisor	MMUs for spatial independence L4 cache partitioning or disabled SAFEXPLAIN SW Stack
Non-AI subsystem and safety control (safety function)	Safety traditional SW	SPE in lockstep configuration No sharing of caches with CCPLEX	CCPLEX CPU Cluster (Cortex A78) in lockstep – different CPU cluster for safety SW	CCPLEX CPU Cluster (Cortex A78) in lockstep – different CPU cluster for safety SW
L2DM	Safety traditional SW	MMUs for spatial independence Safe RTOS on top of SPEs	MMUs for spatial independence L4 cache partitioning or disabled Safe RTOS on top of hypervisor	MMUs for spatial independence L4 cache partitioning or disabled SAFEXPLAIN SW Stack
Pros		Highest independence: different processing elements, less shared resources, different OS and SW stack.	Better performance while preserving independence: different CPU clusters with partitioning approaches, different OS and SW stack.	Ease of integration, same SW stack for all platform elements.
Cons		Limited performance on SPEs.	Need of a hypervisor.	Less independence, required safety guarantees on the SAFEXPLAIN SW Stack.

Table 3. SP1 to NVIDIA Orin resource allocation and configuration options

- Support for lockstep redundancy. As explained before, core clusters in the CCPLEX and the SPE provide lockstep support. Hence, safety related tasks requiring such redundancy can be deployed on the SPE or the CCPLEX by properly configuring the core cluster where they are run.

Determining the configuration at system design time is crucial to perform the timing analysis and interference mitigation and control. Based on the configuration, the timing and interference bounds will be computed and at runtime, L2DM will check that the configuration corresponds to that defined at design time (e.g., through a CRC check).

5.3 Safety Pattern 2

For SP2 the AI-subsystem becomes a safety-critical component. Therefore, in addition to the mechanisms presented in SP1 to guarantee the independence among safety and non-safety software components, new mechanisms are required

to ensure the correct behaviour of the AI-subsystem. In the defined incremental strategy, it is assumed that in *SP2* the AI-based safety function has a low to medium integrity level (i.e., SIL 1 or SIL 2 according to IEC 61508).

5.3.1 System Architecture. *SP2* consists of the same elements as those in the reference safety architecture, resulting in the same conceptual system architecture as that shown in Fig. 4. Based on the target SIL level, *SP2* requires either medium to high diagnostic coverage (60-90% for SIL 1 and 90-99% for SIL 2) or increasing the Hardware Fault Tolerance (HFT) according to IEC 61508-2. However, achieving this $HFT > 0$ on the same chip has several implications on the hardware silicon level (IEC 61508-2 Annex E) which shall be guaranteed by the platform manufacturer. When the platform does not provide such guarantees, off-chip redundancy would be required, which is the case of *SP3*. For *SP2*, the focus is on $HFT = 0$ with low to medium diagnostic coverage.

Taking this into account, the main components of the reference safety architecture are applied as follows in *SP2*:

- **Diverse Redundancy:** On chip redundancy approaches are considered to improve the safe failure fraction and diagnostic coverage of the AI/ML constituent. From Table 2, diverse redundancy schemes with low or medium Diagnostic Coverage (DC) can be applied in this safety pattern, based on the specific requirements of the application (e.g., DSR_1, DSR_2, DSR_3, DSR_4).
- **Diagnostic and monitoring mechanisms:** Same mechanism as in *SP1* apply, with additional diagnostics for the AI/ML constituent in the *L1DM* further elaborated in Subsection 5.3.2.
- **Supervision function:** A supervisor function according to the description in the reference architecture should also be integrated. In this SP, the runtime explanations generated by the supervision function can also support enhanced automated system monitoring during operation, as well as post-hoc analysis following an incident. As these explanations become part of the safety-critical functionality, the required level of rigor for the applied explainability techniques is higher than in *SP1*.

5.3.2 Diagnostic and monitoring mechanisms. *SP2* applies the hierarchical monitoring approach as follows:

- ***SP2-L1DM:*** *L1DM* diagnostics mechanisms can be selected and evaluated to reach the required diagnostic coverage for *SP2*. In addition, if the system includes application with different integrity levels, independence of execution shall be guaranteed following the same approach as in *SP1*. Similarly, to reduce common cause failures among redundant instances in the AI/ML constituent, their independence shall also be guaranteed. Therefore, *L1DM* will monitor the use of resources and other relevant events for each instance of the redundant architecture and share the information with *L2DM* for a platform level monitoring of all system components.
- ***SP2-L2DM:*** Following the incremental approach, *L2DM* platform level diagnostics shall be applied in the same way as for *SP1*. In this case, the NVIDIA Orin platform resources used for the AI/ML constituent become safety relevant too, which would require additional diagnostics with respect to *SP1* and the most suitable platform configurations shall be adapted too. Fig. 7 presents a configuration option that could be adopted for *SP2*, which could be combined with other solutions shown in *SP1* as described in Table 4.

The following mechanisms are provided by the platform:

- **Independent computing components:** In this case, all safety software can be run in CCPLEX CPUs in lockstep mode, using different clusters for the redundant replicas. Other accelerators or different computing resources could also be used to improve diversity.
- **Spatial independence.** The same support for *SP1* addresses the needs of *SP2*.

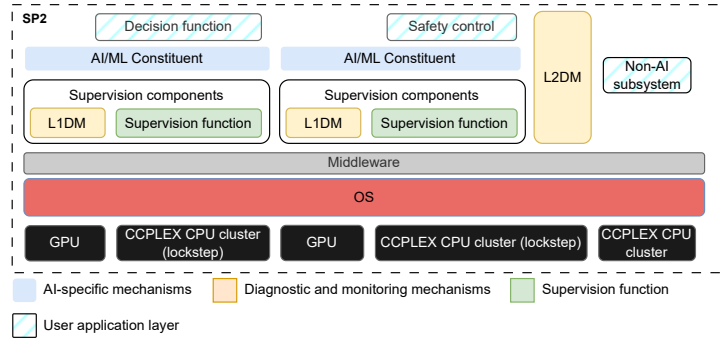


Fig. 7. SP2 to NVIDIA Orin resource allocation and configuration options

SP2 Element	Safety / non-safety	SP2 - A NVIDIA Orin resources and configuration
AI/ML constituent	AI based safety SW	Two instances, each in one separate CCPLX CPU Cluster (Cortex A78) in lockstep configuration GPU for AI inference (depending on the DRS CPU or other computing resources could also be used to improve diversity) Memory controller fabric and traffic from CPU cluster to GPU MMUs for spatial independence Safe separation middleware
Supervision components	Traditional or AI based safety SW	Each AI/ML constituent has each own L1DM and optionally each own supervisor function (depends on user application). Depending on the implementation of the supervision component, it may need GPUs for improved performance (e.g., AI based supervision function). The supervision components can share same CCPLX CPU Cluster (Cortex A78) in lockstep configuration as the AI/ML constituent. MMUs for spatial independence Safe separation middleware
Decision function	Safety traditional SW	These SW components can run on any of the CCPLX CPU Cluster (Cortex A78) in lockstep configuration used for the AI/ML constituent with the same configuration assuming they have the same integrity level.
Safety control	Safety traditional SW	
L2DM	Safety traditional SW	
Non-AI subsystem	Non-safety traditional SW	CCPLX CPU Cluster (Cortex A78) or SPE (no need for lockstep configuration). MMUs for spatial independence L4 cache partitioning or disabled Safe separation middleware or different OS on top of SPEs or hypervisor

Table 4. SP2 to NVIDIA Orin resource allocation and configuration options

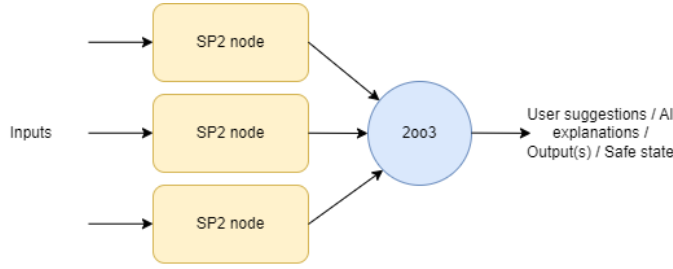


Fig. 8. Conceptual System Architecture - Safety Pattern 3

- Temporal independence. The same support for non-AI tasks for *SP1* addresses the needs of *SP2*. Regarding AI tasks, they are expected to use the GPU and/or the AI accelerators. Those accelerators have been devised to maximize throughput. However, either they do not support concurrency (e.g., AI accelerators) or support it without specific temporal independence support (e.g., GPU). Hence, AI tasks needing concurrency will have a high degree of temporal isolation if they use different accelerators. If they are deployed on the same one (i.e., the GPU) and made run simultaneously, timing interference can be arbitrarily high making its monitoring unfeasible. Therefore, those tasks need to be run sequentially if they need the very same accelerator.
- Support for lockstep redundancy. The same support for non-AI tasks for *SP1* addresses the needs of *SP2*.

5.4 Safety Pattern 3

Safety pattern 3 follows the same approach as SP2 but it is assumed that the system has a higher degree of autonomy and therefore the safety implications of the AI-based subsystem are higher. Therefore, we consider a safety integrity level of SIL 3 / SIL 4 according to IEC 61508 depending on the application.

5.4.1 System Architecture. In order to achieve higher integrity levels (with respect to SP2), SP3 involves either providing a high diagnostic coverage ($\geq 99\%$) or increasing the HFT according to IEC 61508-2. However, achieving a $\geq 99\%$ DC in complex platforms such as the NVIDIA Orin is very challenging and would highly depend on the guarantees provided by the platform manufacturer. The complexity of the Software (SW) stack and lack of transparency on the platform further exacerbates this. Therefore, we assume that for *SP3* an $HFT > 0$ is required. To this end, off-chip redundancy shall be applied due to the restrictions of IEC 61508-2 Annex E as already mentioned in *SP2*. Fig. 8 shows an example of *SP3* implementation based on a TMR architecture, depending on the implementation $HFT = 1$ (Two-out-of-three (2oo3)) or $HFT = 2$ (One-out-of-three (1oo3)) can be achieved.

If we look at each individual replica of the TMR implementation, each node could be implemented following *SP2* safety pattern:

- Diverse Redundancy: In *SP3* off-chip redundancy is applied. Therefore, the decision function should combine the outputs of different computing nodes. Each computing node can have its own decision function in such a way that if any node detects discrepancies, the safe state could be reached ($1oo3 - HFT = 2$). Alternatively, the voting can be done in an external element (e.g., *L3DM*) and apply majority voting, meaning that at least 2 of the 3 redundant nodes shall provide the same output ($HFT = 1$). In order to apply this off-chip redundancy, diverse redundancy schemes with medium to high DC can be applied in this safety pattern, based on the specific requirements of the application (e.g., DSR_4, DSR_5, DSR_6). Off-chip redundancy can be combined with on-chip

redundancy approaches to improve the safe failure fraction and diagnostic coverage of each redundant node, following the same solutions as in *SP2*.

- Diagnostic and monitoring mechanisms: In each redundant instance, the same solutions as for *SP2* apply in *SP3*. *L3DM* would now monitor the status of all redundant replicas.
- Supervision function: Each redundant replica of *SP3* can implement a supervisor function according to the description in the reference architecture in Section 4.2.3, including the explainability techniques for enhanced system supervision in the same way as in *SP2*.
- Traditional subsystem: depending on the application, a fallback subsystem implemented with traditional SW could be in place.

6 Railway Automatic Train Operation System

To illustrate the applicability of the defined reference safety architecture pattern, we consider a railway use case involving an Automatic Train Operation (ATO) system and we provide an application example for one of the safety patterns. This system leverages AI-based perception to detect obstacles on the tracks and take appropriate actions to mitigate risks such as collisions or damage to passengers and infrastructure. The ATO system uses stereo cameras (left and right) to identify potential hazards in the train's path. Through a combination of object detection, classification, and distance estimation, the system categorizes obstacles based on their criticality and position relative to the track.

6.1 Incremental strategy for ATO system

Depending on the operational mode and the AI's level of responsibility, the ATO system reacts by issuing driver warnings, engaging the service brake, or activating the emergency brake, achieving different Grades of Automation (GoA):

- DL Usage level D [*SP1*]: The train operates under full human control (GoA 1). The AI subsystem acts purely as an advisory module with no direct influence on train control. As an output, the AI module provides alerts to the driver. This scenario should be addressed by *SP1*.
- DL Usage level C [*SP1*]: The human driver has the final responsibility of commanding the brakes in case of obstacle detection, but the DL subsystem can provide assistance for instance by automatically commanding a reduction of the speed. While the DL subsystem does not participate in the safety function of braking, it can have an impact on it. This scenario should be addressed by *SP1*.
- DL Usage level A2 [*SP2*]: The train operates in semi-automated mode (GoA 2), where both the AI system and the human driver share responsibility for ensuring operational safety. The AI subsystem contributes to safety by triggering service braking in response to critical obstacles, while the human operator retains ultimate control.
- DL Usage level A1 [*SP3*]: The AI subsystem assumes full responsibility for the braking function (GoA 3-4). The AI system autonomously detects obstacles, estimates braking distance, and makes real-time decisions to activate either the service or emergency brake, depending on collision proximity.

6.2 GoA 2 semi-automated train operation – Safety Pattern 2 (SP 2)

On the GoA2 semi-automated train operation use-case, the object and track detection AI-based function participates in the control of the service brake, which corresponds to SIL 2 safety function. Taking this into account, the main components of the railway ATO system are mapped as shown in Fig. 9 in *SP2*:

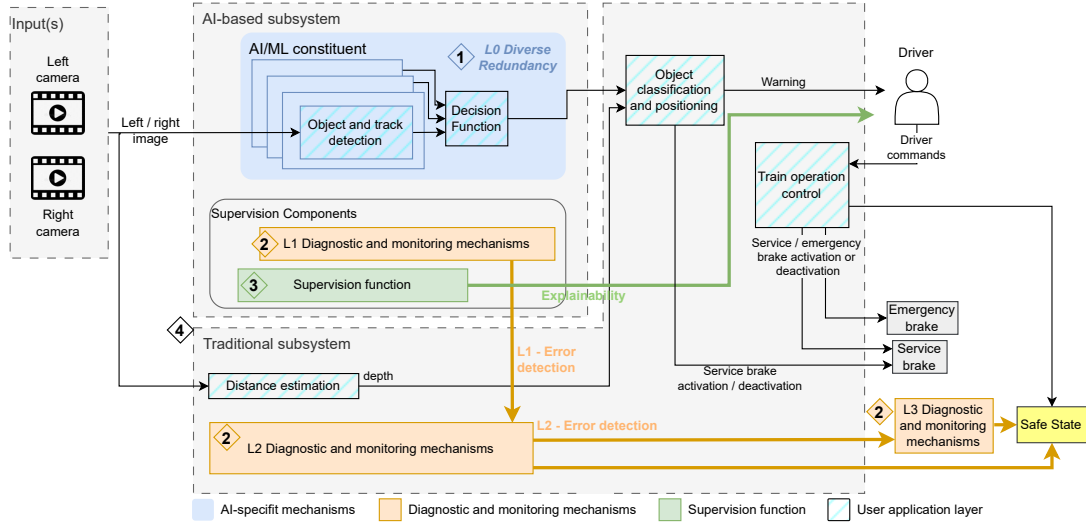


Fig. 9. SP2 instantiation for a GoA 2 ATO system

6.2.1 *AI-based subsystem.* Includes the AI-based object and track detection function, based on AI vision algorithms. Safety of such element is supported by:

- **Diverse Redundancy:** On chip redundancy approaches are considered to improve the safe failure fraction and diagnostic coverage of the object and track detection function running in the AI/ML constituent.
- **Diagnostic and monitoring mechanisms:** Same mechanism as in SP1 apply, with additional diagnostics for the object and track detection function running in the AI/ML constituent in the L1DM.
- **Supervision function:** A supervision function according to the description in the reference architecture shall be integrated too.

6.2.2 *Traditional subsystem.* The traditional subsystem comprises the distance estimation and object classification and positioning SIL 2 safety functions, together with the train operation control. These components are complemented with the L2DM for monitoring platform level errors and communicating with the external L3DM module.

7 Related Work

Integrating EAI into safety-critical systems necessitates adherence to established functional safety standards to ensure these systems operate reliably and safely. Recent advancements have focused on developing frameworks and methodologies to address the unique challenges posed by EAI in this context. Considerable efforts are devoted to standardization frameworks, with new functional safety standards such as ISO/IEC TR 5469 [14], ISO/PAS 8800 [6], ANSI/UL 4600 [4], ISO 21448 [3] and VDE-AR-2842-61 [5] that focus on ensuring AI systems operate safely within their intended contexts. Compliance involves conducting risk analysis, developing a safety plan to mitigate identified risks, and collecting evidence of adherence to the standards. Adhering to these frameworks and practices is essential for organizations aiming to develop and deploy AI in critical systems. However, these standards do not yet define specific safety mechanisms to address runtime safety challenges. Instead, they provide guidance on the architectural elements

that should be considered. In this work, we take these architectural considerations as a baseline to define a generic, reusable safety architecture and extend them with specific safety mechanisms applied at each level, facilitating the safe integration of AI into critical systems.

Preliminary research has explored the alignment of AI with emerging standards, such as ANSI/UL 4600, ISO 21448, and ISO 26262. In the context of ANSI/UL 4600, researchers in [12] define an auditable safety case within the aerospace sector. Similarly, a study on ISO 21448 by [13] highlights the potential necessity of supervisory mechanisms to ensure AI safety. Other studies focus on the ISO 26262 software requirements, proposing adaptations for AI systems [21–24]. The authors in [15] further identify key safety challenges associated with AI and propose a requirements model, along with an evaluation approach for ensuring conformity. Additionally, numerous works address specific safety challenges introduced by AI, which have been systematically surveyed and categorized at different hierarchical levels by Perez et al. [20]. However, due to the novelty of ISO/IEC TR 5469 and ISO/PAS 8800, to the best of our knowledge, this is the first work to bridge the gap by proposing compliant architectural designs, an essential step in the development of any safety-critical system.

8 Conclusions and Future Work

This paper presents a modular reference architecture pattern for safety-critical EAI systems, addressing the unique challenges of AI integration. Defining a system architecture is a crucial step in safety system development, as it establishes the structural framework, functional interactions, and safety mechanisms necessary to ensure reliability, regulatory compliance, and effective risk mitigation throughout the system lifecycle. By aligning with functional safety and AI standards such as ISO/IEC TR 5469 and ISO/PAS 8800, the proposed architecture bridges the gap between safety and AI by providing a structured framework that integrates safety principles with AI-driven functionalities. The reference safety architecture serves as a reusable foundation, supporting the progressive adoption of EAI in safety-critical domains through an incremental approach. This work introduces multiple safety mechanisms, including diverse redundancy schemes, diagnostic and monitoring mechanisms, and a supervision function, all of which enhance diagnostic coverage and mitigate AI-specific risks, promoting the safe deployment of AI-driven systems. The practical adaptability of the reference architecture is demonstrated through its application to three safety patterns on the NVIDIA Jetson AGX Orin platform. This incremental approach allows experience gained from lower-criticality implementations to improve and refine solutions for higher-criticality applications, fostering a structured and confidence-building transition toward AI integration in safety-critical systems. Although this work does not include a quantitative evaluation of the proposed safety reference architecture, a qualitative assessment has been carried out through review sessions with two independent certification authorities. These reviewers provided expert feedback on the completeness, clarity, and practical relevance of the architectural patterns with respect to safety standards and certification practices. This validation helped shape the architecture to ensure its applicability in real-world safety-critical AI systems. Nevertheless, future work will focus on complementing this with quantitative evaluations to assess the architecture’s effectiveness in specific use case instantiations.

Therefore, future work will focus on applying the proposed architecture and strategy to specific use cases to validate its effectiveness in real-world safety-critical scenarios. Additionally, further research will be dedicated to developing and evaluating specific safety mechanisms that support the implementation of the proposed safety patterns. This includes assessing their performance and suitability using quantitative metrics such as DC together with fault injection to determine their appropriateness for different SILs. In addition, we acknowledge the critical importance of cybersecurity and its intersection with functional safety, particularly in addressing adversarial threats and model

robustness. Integrating security considerations into the proposed architecture is therefore identified as an essential direction for future work.

Acknowledgments

The research leading to these results has received funding from the SAFEXPLAIN project of the European Union's Horizon Europe programme under grant agreement number 101069595 and from CAPSUL-IA project PLEC2023-010240 funded by the Spanish Ministry of Science and Innovation (MICIU/AEI/10.13039/501100011033).

References

- [1] 2010. IEC 61508(-1/7): Functional safety of electrical / electronic / programmable electronic safety-related systems.
- [2] 2018. ISO 26262(-1/11) Road vehicles – Functional safety.
- [3] 2019. ISO/PAS 21448 Road vehicles – Safety of the intended functionality.
- [4] 2020. ANSI/UL 4600 Standard for Safety for the Evaluation of Autonomous Products.
- [5] 2020. VDE-AR-E 2842-61 - Design and Trustworthiness of autonomous/cognitive systems.
- [6] 2023. ISO/CD PAS 8800 Road Vehicles — Safety and artificial intelligence.
- [7] Certification Authorities Software Team (CAST). 2022. CAST-32A position paper.
- [8] Möllmann Ertl, Rehm Nägler, and Veldten Staib. [n. d.]. Standardized E-Gas Monitoring Concept for Gasoline and Diesel Engine Control Units V5. 5. *EGAS working group* ([n. d.]).
- [9] European Union Aviation Safety Agency (EASA). 2022. AMC 20-193 Use of multi-core processors.
- [10] European Union Aviation Safety Agency (EASA). 2023. EASA Concept Paper: guidance for Level 1 & 2 machine learning applications.
- [11] Javier Fernández et al. 2024. AI-FSM. <https://doi.org/10.5281/zenodo.10964402>
- [12] U. D. Ferrell and A. H. A. Anderegg. 2020. Applicability of UL 4600 to Unmanned Aircraft Systems (UAS) and Urban Air Mobility (UAM). In *AIAA/IEEE 39th Digital Avionics Systems Conference (DASC)*. 1–7. <https://doi.org/10.1109/DASC50938.2020.9256608>
- [13] J. Henriksson, C. Berger, M. Borg, L. Tornberg, S. R. Sathyamoorthy, and C. Englund. 2019. Performance Analysis of Out-of-Distribution Detection on Various Trained Neural Networks. In *2019 45th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. 113–120. <https://doi.org/10.1109/SEAA.2019.00026>
- [14] ISO/IEC. [n. d.]. ISO/IEC CD TR 5469 Artificial intelligence — Functional safety and AI systems.
- [15] C. Klüver, A. Greisbach, M. Kindermann, and B. Püttmann. 2024. A requirements model for AI algorithms in functional safety-critical systems with an explainable self-enforcing network from a developer perspective. *Security and Safety* 3 (2024), 2024020. <https://doi.org/10.1051/sands/2024020>
- [16] Shaoshan Liu and Shuang Wu. 2024. *A Brief History of Embodied Artificial Intelligence and Its Future Outlook*. <https://cacm.acm.org/blogcacm/a-brief-history-of-embodied-artificial-intelligence-and-its-future-outlook/> Communications of the ACM Blog.
- [17] Robert Lowe. 2024. Explainable AI for Systems with Functional Safety Requirements. http://safexplain.eu/wp-content/uploads/2025/02/2024_XAI_HIPEAC_webinar_RISE.pdf
- [18] NVIDIA. [n. d.]. NVIDIA Jetson AGX Orin – Embedded Systems for Autonomous Machines. <https://www.nvidia.com/en-eu/autonomous-machines/embedded-systems/jetson-orin/>. Accessed: 2025-07-22.
- [19] Rafael Padilla, Sergio L Netto, and Eduardo AB Da Silva. 2020. A survey on performance metrics for object-detection algorithms. In *2020 international conference on systems, signals and image processing (IWSSIP)*. IEEE, 237–242.
- [20] Jon Perez-Cerrolaza, Jaume Abella, Markus Borg, Carlo Donzella, Jesús Cerquides, Francisco J. Cazorla, Cristofer Englund, Markus Tauber, George Nikolakopoulos, and Jose Luis Flores. 2023. Artificial Intelligence for Safety-Critical Systems in Industrial and Transportation Domains: A Survey. *ACM Comput. Surv.* (2023). <https://doi.org/10.1145/3626314>
- [21] Rick Salay and Krzysztof Czarnecki. 2018. Using Machine Learning Safely in Automotive Software: An Assessment and Adaption of Software Process Requirements in ISO 26262. *arXiv preprint arXiv:1808.01614* (2018).
- [22] Rick Salay, Rodrigo Queiroz, and Krzysztof Czarnecki. 2018. An Analysis of ISO 26262: Using Machine Learning Safely in Automotive Software. *ArXiv abs/1709.02435* (2018).
- [23] H. Tabani, L. Kosmidis, J. Abella, F. J. Cazorla, and G. Bernat. 2019. Assessing the Adherence of an Industrial Autonomous Driving Framework to ISO 26262 Software Guidelines. In *2019 56th ACM/IEEE Design Automation Conference (DAC)*. 1–6.
- [24] H. Tabani, R. Pujol, J. Abella, and F. J. Cazorla. 2020. A Cross-Layer Review of Deep Learning Frameworks to Ease Their Optimization and Reuse. In *IEEE 23rd International Symposium on Real-Time Distributed Computing (ISORC)*. 144–145.

Received 03 April 2025; revised 31 July 2025; accepted 8 September 2025